

PROJECT REPORT

AI Driver Drowsiness & Distraction Monitor

A report submitted in partial fulfilment of the requirements for the

VITyarthi Flipped Course Evaluation

in

COMPUTER VISION / REAL-TIME SYSTEMS

by

MURUGAVEL J

Reg. No: 23MIM10074

Integrated M.Tech (Artificial Intelligence)

**SCHOOL OF COMPUTING SCIENCE ENGINEERING AND ARTIFICIAL INTELLIGENCE
(SCAI)**

VIT BHOPAL UNIVERSITY

2026

DECLARATION

I, Murugavel J (Reg. No: 23MIM10074), declare that the project report titled “AI Driver Drowsiness & Distraction Monitor”, submitted for the VITyarthi Flipped Course Evaluation in the Computer Vision and Real-Time Systems domain, is my own work. I carried it out as part of the Integrated M.Tech (Artificial Intelligence) programme at the School of Computing Science Engineering and Artificial Intelligence, VIT Bhopal University. I have not submitted this work anywhere else for any other degree or diploma.

MURUGAVEL J
(23MIM10074)

ABSTRACT

The Domain

This project sits in the Computer Vision and Real-Time Systems area. The question I wanted to answer was simple: can you tell whether a driver is falling asleep or looking away from the road using nothing but an ordinary webcam? No infrared sensors, no steering-wheel hardware, and no deep learning model trained from scratch.

The Problem

Tired and distracted drivers cause a huge share of road accidents. In the United States alone, the National Highway Traffic Safety Administration links roughly 91,000 crashes and about 800 deaths every year to drowsy driving. Systems that watch the driver already exist, but most of them ship with dedicated hardware, which pushes the price up and keeps the feature inside premium vehicles. I wanted to see how far a purely software solution could go on a normal laptop with whatever camera it already has.

Methodology

The system builds on Google's pre-trained MediaPipe Face Mesh model, which returns 468 3D facial landmarks per frame and runs at around 30 FPS on a CPU. From those landmarks I compute three classical metrics. The Eye Aspect Ratio (EAR) tells me how open the eyes are, so it covers blinking, sustained eye closure and microsleep. The Mouth Aspect Ratio (MAR) picks up yawns. Head pitch, yaw and roll come from OpenCV's solvePnP and tell me whether the driver is nodding off or turning to look at something else.

These continuous numbers feed a small state machine with four levels, ALERT, CAUTION, WARNING and DANGER, which decides when to play an audio alert through pygame. A two-second cooldown stops the alerts from becoming background noise. Every frame's metrics also go into a CSV file so the whole session can be reviewed afterwards.

Key Findings

The report walks through the full pipeline: the EAR and MAR formulas and the thresholds I settled on after testing, the six-point solvePnP head model, the four-level alert logic, and how the code is split across monitor.py, the utils package, analyze_session.py and demo_simulate.py. The main takeaway is that landmark-based classical metrics, with zero training on my part, are good enough to catch drowsiness and distraction in real time on CPU-only hardware. The trade-off is that accuracy depends a fair bit on lighting, eyewear and where the camera sits.

Outcome

What came out of it is a working real-time driver safety monitor that needs nothing more than a webcam. It shows a live HUD, escalates its alerts as the situation gets worse, logs every frame, and includes a demo mode that runs without a camera at all. The analytics script then turns a session log into charts of EAR and MAR trends, head-pose variation and the alert-level timeline.

TABLE OF CONTENTS

Chapter No.	Section	Page No.
1	Project Objectives	1
2	Introduction & Problem Statement	2
3	System Architecture	3
4	Methodology	4
	4.1 Eye Aspect Ratio (EAR)	4
	4.2 Mouth Aspect Ratio (MAR)	5
	4.3 Head Pose Estimation (solvePnP)	5
	4.4 Alert State Machine	6
5	Implementation & Module Design	7
6	Technology Stack	9
7	Results & Analysis	10
8	Limitations	12
9	Future Enhancements	12
10	Conclusion	13
	References	14

CHAPTER 1 — PROJECT OBJECTIVES

I took this project up as part of the VITyarthi Flipped Course Evaluation in the Computer Vision and Real-Time Systems domain. These were the goals I set at the start:

- Build a real-time driver drowsiness and distraction monitor that works from a plain webcam feed and nothing else.
- Use a pre-trained facial landmark model, MediaPipe Face Mesh, instead of training my own network, so the whole thing stays light enough to run on a CPU.
- Catch three kinds of unsafe behaviour: eyes closing or microsleep, yawning, and the head nodding or turning away.
- Make the alerts escalate. A brief droop should not sound the same as a two-second microsleep.
- Log every frame and write an analysis tool that turns a session into readable charts.
- Keep everything runnable from the command line, and add a demo mode so an evaluator without a webcam can still see the full pipeline work.

CHAPTER 2 — INTRODUCTION & PROBLEM STATEMENT

Fatigue and distraction are among the biggest causes of road accidents anywhere in the world. NHTSA figures put drowsy driving behind roughly 91,000 crashes and around 800 deaths a year in the United States by itself. Driver-monitoring systems do exist in the market, but they usually rely on a dedicated infrared camera or sensors built into the steering wheel. That drives up the cost and keeps the feature restricted to expensive cars.

My proposal here is the software-only version. Everything runs off a standard laptop webcam feed. There is no extra hardware to buy and no labelled dataset to collect, because the perception work is handled by Google's pre-trained MediaPipe Face Mesh model.

Key Goals

- Detect eye drowsiness, microsleep, yawning and head-pose deviation while the car is moving.
- Raise audio alerts in three escalating levels: Caution, then Warning, then Danger.
- Keep logging throughout the trip so the session can be reviewed later.
- Run from the command line with almost no setup and no GPU requirement.

CHAPTER 3 — SYSTEM ARCHITECTURE

The pipeline is linear. A frame comes in from the webcam or a video file, and by the end of the same frame the system has updated the on-screen HUD, possibly fired an audio alert, and written a row to the session log.

High-level data flow

- Webcam or video file supplies frames at roughly 30 FPS.
- MediaPipe Face Mesh pulls 468 3D facial landmarks out of each frame, entirely on the CPU.
- Two calculations branch off those landmarks: EAR and MAR from eye and mouth geometry, and head pitch, yaw and roll from solvePnP.
- Both streams go into a time-based alert state machine with four states: ALERT, CAUTION, WARNING and DANGER.
- The state machine drives two outputs at once, an OpenCV HUD overlay and a pygame audio alert playing on a background thread.
- In parallel, every frame's metrics are appended to a CSV log for offline analysis.

The useful part of this layout is that perception, decision-making and output stay separate. Each stage can be tested on its own. `demo_simulate.py` takes advantage of that directly: it swaps the entire perception stage for synthetic data while the state machine, HUD and logger carry on unchanged.

CHAPTER 4 — METHODOLOGY

4.1 Eye Aspect Ratio (EAR)

EAR is worked out separately for each eye from six landmark points placed around it, using the formula Soukupová and Čech published in 2016:

$$EAR = (\|p2 - p6\| + \|p3 - p5\|) / (2 \times \|p1 - p4\|)$$

An open eye lands around 0.30 and a fully closed eye drops to about 0.05, which is a wide enough gap to be reliable. The left eye uses landmark indices [362, 385, 387, 263, 373, 380] and the right eye uses [33, 160, 158, 133, 153, 144] from the 468-point mesh.

Threshold: an EAR below 0.25 held for at least 15 consecutive frames, which is about half a second, triggers a CAUTION event.

4.2 Mouth Aspect Ratio (MAR)

MAR compares how far the mouth is open vertically against its horizontal width, following Abtahi et al. (2014):

$$MAR = vertical_opening / horizontal_width$$

Eight mouth landmarks feed into this: [61, 291, 39, 181, 0, 17, 269, 405].

Threshold: a MAR above 0.65 held for at least 20 frames, roughly 0.65 seconds, is recorded as a yawn.

4.3 Head Pose Estimation (solvePnP)

For head orientation I use OpenCV's solvePnP. It takes six 2D facial landmarks, the nose tip, the chin and the four eye and mouth corners, and matches them against a generic 3D face model measured in millimetres. The rotation matrix that comes back is then broken down into Euler angles using cv2.decomposeProjectionMatrix.

Landmark	3D Model Point (mm)
Nose tip	(0, 0, 0)
Chin	(0, -330, -65)
Left eye corner	(-225, 170, -135)
Right eye corner	(225, 170, -135)
Left mouth corner	(-150, -150, -125)
Right mouth corner	(150, -150, -125)

- Pitch above 20° means the head is nodding downward, which reads as drowsy.
- Yaw above 35° means the driver is looking away, which reads as distracted.

4.4 Alert State Machine

The continuous EAR, MAR and pose signals need to become something a driver can actually respond to, so a time-based state machine folds them into four escalating states:

State	Trigger	Level	Colour
ALERT	Everything normal	0	Green
CAUTION	EAR low for ~0.5s, or head deviation	1	Yellow
WARNING	EAR low for ~1.0s, or yawning	2	Orange
DANGER	EAR low for ~2.0s (microsleep)	3	Red

The frame counts behind these are `CONSEC_FRAMES_CAUTION` = 15 (about 0.5s), `CONSEC_FRAMES_WARNING` = 30 (about 1.0s) and `CONSEC_FRAMES_DANGER` = 60 (about 2.0s, the usual microsleep cut-off). Audio plays on a background thread with a two-second cooldown so the driver does not start tuning the beeps out, and the intensity scales with severity, from one beep at CAUTION up to four at DANGER.

CHAPTER 5 — IMPLEMENTATION & MODULE DESIGN

5.1 monitor.py — Core Detection Engine

This is the entry point. It grabs frames through `cv2.VideoCapture` from either a webcam or a video file, runs MediaPipe Face Mesh on each one, computes EAR, MAR and the three pose angles, updates the alert state machine, and draws the HUD with metric bars, a head-direction compass and blink and yawn counters. Every frame also goes to `SessionLogger`. While it is running you can press `q` to quit, `s` to save a screenshot, `r` to reset the counters, and `c` to toggle the landmark mesh overlay.

5.2 utils/face_metrics.py — Metric Computations

A pure computation module with no side effects, which makes it easy to test. It holds `eye_aspect_ratio()`, `mouth_aspect_ratio()` and `head_pose_angles()`, the last of which chains `solvePnP`, `Rodrigues` and `decomposeProjectionMatrix`. The landmark index constants for the 468-point mesh live here too.

5.3 utils/alert_system.py — Alert Manager

A thread-safe alert manager. It handles the two-second cooldown, synthesises sine-wave beeps on the fly through `pygame` so there are no audio files to ship, keeps three distinct alert levels at one, two and four beeps, and plays everything on a background thread so audio never stalls the video loop.

5.4 utils/session_logger.py — Data Logger

A CSV logger in append mode. For each frame it records `timestamp`, `elapsed_sec`, `ear_left`, `ear_right`, `ear_avg`, `mar`, `pitch`, `yaw`, `roll`, `status`, `alert_level` and `event`.

5.5 analyze_session.py — Post-Session Analytics

This reads a session log and produces a multi-panel `matplotlib` figure: EAR over time with blink markers, MAR over time with yawn markers, the three head-pose angles, the alert-level timeline, a status-distribution pie chart, and a block of summary statistics covering total blinks, total yawns and time spent at each alert level.

5.6 demo_simulate.py — Webcam-Free Demo

This writes a realistic synthetic session log: sinusoidal EAR signals with blink events dropped in, MAR pulses standing in for yawns, and head-pose drift with occasional distraction spikes. Anyone evaluating the project without a camera can still run the analytics end to end.

5.7 Project Structure

`AI-Driver-Drowsiness-Distracton-Monitor/` contains `monitor.py`, `analyze_session.py`, `demo_simulate.py`, `requirements.txt`, `README.md` and `REPORT.md` at the top level. The `utils/` package holds `face_metrics.py`, `alert_system.py`, `session_logger.py` and `visualize_landmarks.py`. Two folders, `logs/` and `plots/`, are created automatically for session CSVs and analysis charts.

CHAPTER 6 — TECHNOLOGY STACK

Component	Technology
Programming Language	Python 3.10 (works on 3.8–3.11)
Face Landmark Detection	MediaPipe Face Mesh (Google, pre-trained)
Image Processing	OpenCV 4.8
Numerical Computation	NumPy, SciPy
Audio Alerts	pygame 2.5
Session Logging	Python csv module
Data Analysis	pandas, matplotlib, seaborn
Head Pose Estimation	OpenCV solvePnP

CHAPTER 7 — RESULTS & ANALYSIS

7.1 Real-Time Monitor Output

With `monitor.py` running, the HUD puts the current state across the top banner in its matching colour. A sidebar on the left shows EAR for the left eye, right eye and average, plus MAR, each drawn as a bar with its threshold marked. A small 2D compass shows which way the head is pointing, and the corner carries the session timer, FPS, blink count and yawn count.

7.2 Threshold Validation

Metric	Normal Range	Alert Threshold
EAR (open eye)	0.28 – 0.35	< 0.25
EAR (closed eye)	0.05 – 0.15	—
MAR (closed mouth)	0.15 – 0.30	—
MAR (yawning)	> 0.65	> 0.65
Head pitch	-10° to +10°	> 20°
Head yaw	-15° to +15°	> 35°

7.3 Post-Session Analytics

`analyze_session.py` saves a six-panel matplotlib figure into `plots/`: the EAR timeline with blink events, the MAR timeline with yawn events, the head-pose angle plots, the alert-level timeline, a status-distribution pie chart and a summary-statistics table. Across my test runs the system held roughly 25 to 30 FPS on CPU-only hardware, so the real-time requirement is met without touching a GPU.

CHAPTER 8 — LIMITATIONS

A few things the system does not handle well, which are worth stating plainly:

- Only one face. It locks onto the first face it finds, so scenarios with more than one person in frame are out of scope.
- Lighting matters. MediaPipe's landmark accuracy falls off in very dim light or when the frame is blown out.
- Glasses can cause trouble. Reflective lenses interfere with eye-landmark detection.
- Camera position is fixed. The camera has to be roughly front-facing; a side-mounted camera will not work.
- No GPU acceleration. The pipeline was written for CPU, though a GPU-accelerated MediaPipe build would speed it up.

CHAPTER 9 — FUTURE ENHANCEMENTS

- Add PERCLOS, the percentage of time the eyes are closed, for a more clinically grounded drowsiness score.
- Track blink rate per minute as a second fatigue indicator.
- Support night driving through an infrared camera.
- Port to Android and iOS using TensorFlow Lite.
- Build a web dashboard so a fleet operator can watch several drivers at once.
- Layer a small deep-learning classifier on top of the existing landmark features.

CHAPTER 10 — CONCLUSION

This project shows that a working real-time driver safety monitor can be built with nothing more than an ordinary webcam and a normal laptop. Combining Google's pre-trained MediaPipe Face Mesh landmarks with three classical computer-vision measures, Eye Aspect Ratio, Mouth Aspect Ratio and solvePnP head pose, was enough to detect drowsiness, yawning and distraction reliably, and I never had to collect a dataset or train a model.

The four-level alert state machine and the per-frame CSV logging are what push it past being just a demo. The alerts escalate the way a real safety system should, and the logs make a trip reviewable after the fact. Everything runs from a couple of command-line instructions, and I deliberately included `demo_simulate.py` so the whole pipeline, from data generation through to the analytics charts, can be checked by anyone who does not have a camera in front of them.

Submitted by Murugavel J (Reg. No: 23MIM10074) for the VITyarthi Flipped Course Evaluation, Computer Vision domain, VIT Bhopal University.

REFERENCES

1. Kartynnik, Y. et al. "Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs." CVPR Workshop, 2019. (MediaPipe Face Mesh)
2. Soukupová, T. and Čech, J. "Real-Time Eye Blink Detection using Facial Landmarks." CVWW, 2016. (EAR for drowsiness detection)
3. OpenCV Documentation. `cv2.solvePnP`, `cv2.decomposeProjectionMatrix`. (Head pose estimation)
4. Abtahi, S. et al. "YAWNNet: Detecting Yawning and Pose in the Wild." 2014. (MAR for yawn detection)
5. National Highway Traffic Safety Administration (NHTSA). Drowsy Driving Statistics, 2017 Data.
6. Project repository: <https://github.com/murugavel03/AI-Driver-Drowsiness-Distracton-Monitor>